

High-Level Design (HLD)

Automated HVAC Duct Detection and Annotation Web Application

1. HLD Objective

This HLD defines the architecture of a **browser-based application** that allows a user to:

1. upload an HVAC mechanical drawing PDF
2. submit it for processing
3. detect ductwork and associated labels
4. measure duct lengths
5. preview annotated results in the browser
6. download annotated PNG/PDF and structured output

The sample drawing indicates a **single-sheet mechanical floor plan**, labeled **M2.0**, for **DHC Midlothian**, with a visible scale and plan layout. That supports a workflow centered on document upload, backend analysis, and annotated drawing generation.

2. Architectural Goals

The architecture should optimize for:

- **web usability**: simple browser-based upload and review
- **asynchronous processing**: drawing analysis may take time
- **modularity**: PDF parsing, duct detection, measurement, and annotation should be separable
- **scalability**: multiple jobs/users should be supported
- **reviewability**: the primary output is a visual markup
- **traceability**: jobs, outputs, and logs should be retrievable
- **extensibility**: future support for other MEP elements

High-Level Design (HLD)

3. High-Level Architecture Overview

The system is composed of six primary layers:

1. **Frontend Web App**
2. **API Gateway / Backend Application**
3. **Job Queue and Worker Layer**
4. **Document Processing Engine**
5. **Storage Layer**
6. **Observability and Admin Layer**

The frontend handles user interaction.

The backend handles authentication, job creation, metadata, and result serving.

The worker layer performs long-running analysis.

The processing engine performs the HVAC-specific logic.

Storage persists files, results, and metadata.

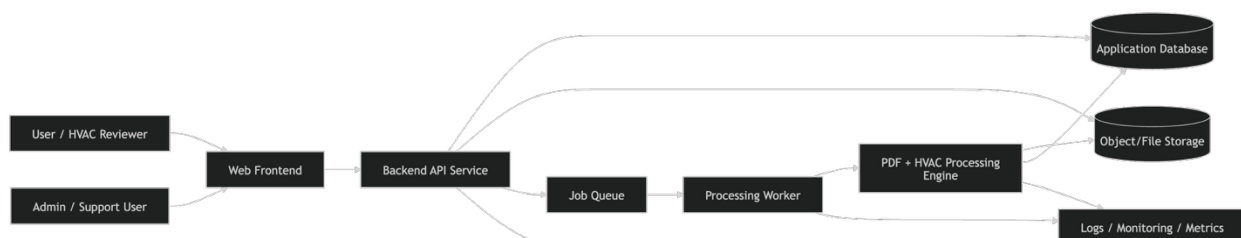
Observability supports debugging and operations.

4. High-Level System Context Diagram

Explanation

At the highest level, the user interacts only with the web app. The web app talks to backend APIs. The backend stores files, creates processing jobs, and delegates analysis to workers. The workers use the processing engine to generate outputs, which are then made available to the user.

Mermaid Diagram



High-Level Design (HLD)

5. Container-Level HLD

Explanation

This view shows the main deployable services.

5.1 Frontend

A browser-based SPA or SSR app, likely React/Next.js, responsible for:

- login
- file upload
- job list
- processing status
- result preview
- output download

5.2 Backend API

A web server, likely FastAPI or Node.js backend, responsible for:

- auth/session integration
- upload orchestration
- job creation
- status retrieval
- result metadata retrieval
- admin endpoints

5.3 Queue

A message broker or task queue such as Redis Queue, Celery, RabbitMQ, or SQS.

5.4 Worker

Consumes queued jobs and performs the heavy processing pipeline.

High-Level Design (HLD)

5.5 Processing Engine

Core analysis modules:

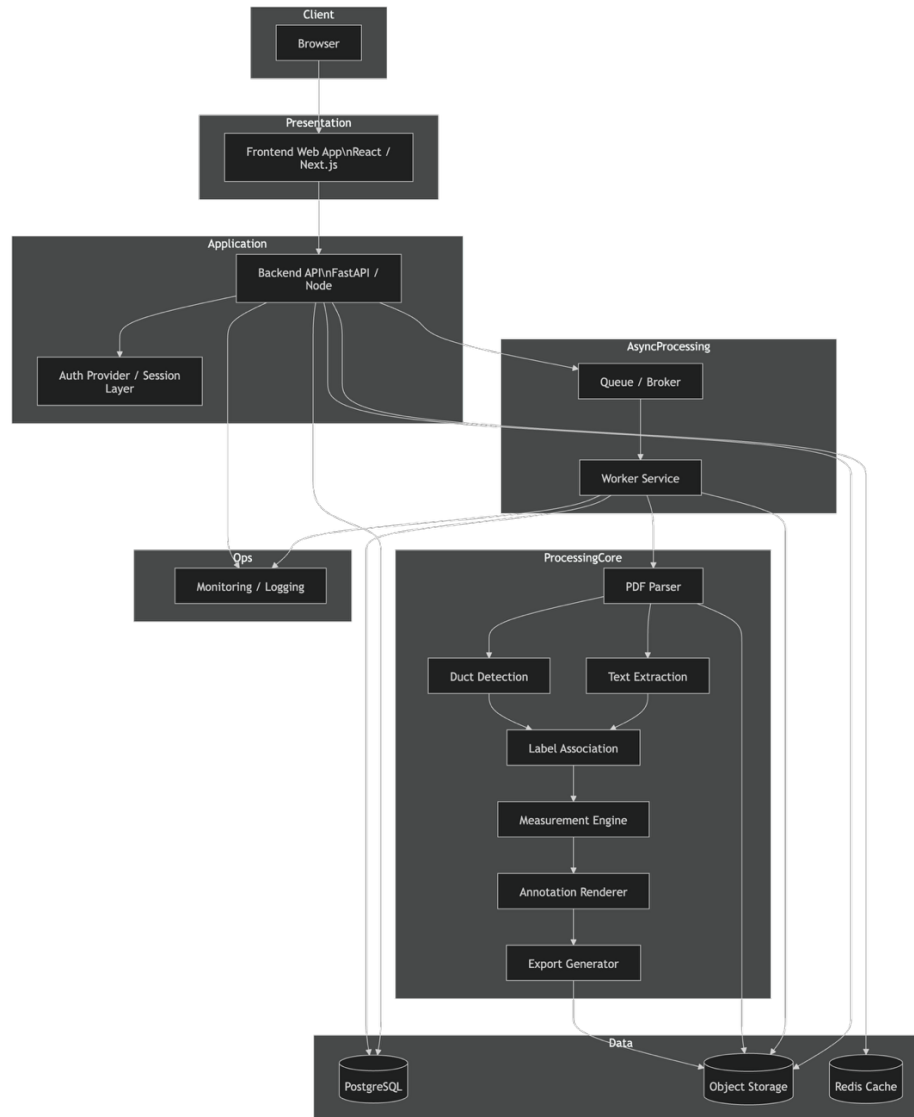
- PDF ingestion
- page rendering
- text extraction
- duct detection
- label association
- measurement
- annotation rendering
- export generation

5.6 Storage

- object storage for source PDFs, rendered images, annotated outputs
- relational DB for users, jobs, page metadata, extracted segments, statuses

High-Level Design (HLD)

Mermaid Diagram



High-Level Design (HLD)

6. Core Processing Pipeline HLD

Explanation

This is the most important application-specific view.

The pipeline is:

1. Load PDF
2. Validate and extract page metadata
3. Render page image
4. Detect main plan region and exclude title block/notes
5. Extract text and coordinates
6. Detect duct geometry
7. Build connected duct graph
8. Match labels to segments
9. Estimate segment lengths
10. Render overlays
11. Save outputs and metadata

The sample input clearly has large non-plan sections such as notes and title regions, so **plan-region isolation** is an important early-stage step.

Mermaid Diagram



High-Level Design (HLD)

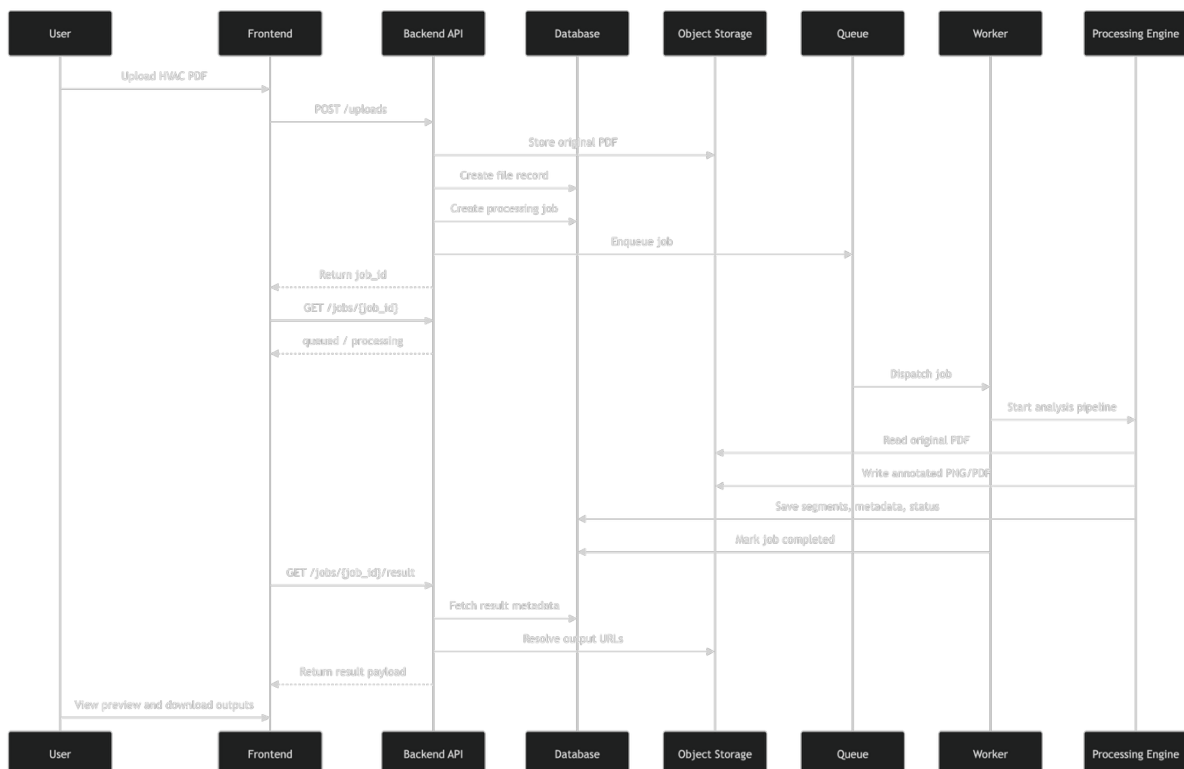
7. Web Application Flow HLD

Explanation

This diagram shows how the user experience maps to system behavior.

1. user uploads PDF
2. backend creates job
3. queue dispatches work
4. worker processes
5. outputs are stored
6. frontend polls or subscribes for status
7. user views results

Mermaid Diagram



High-Level Design (HLD)

8. Deployment HLD

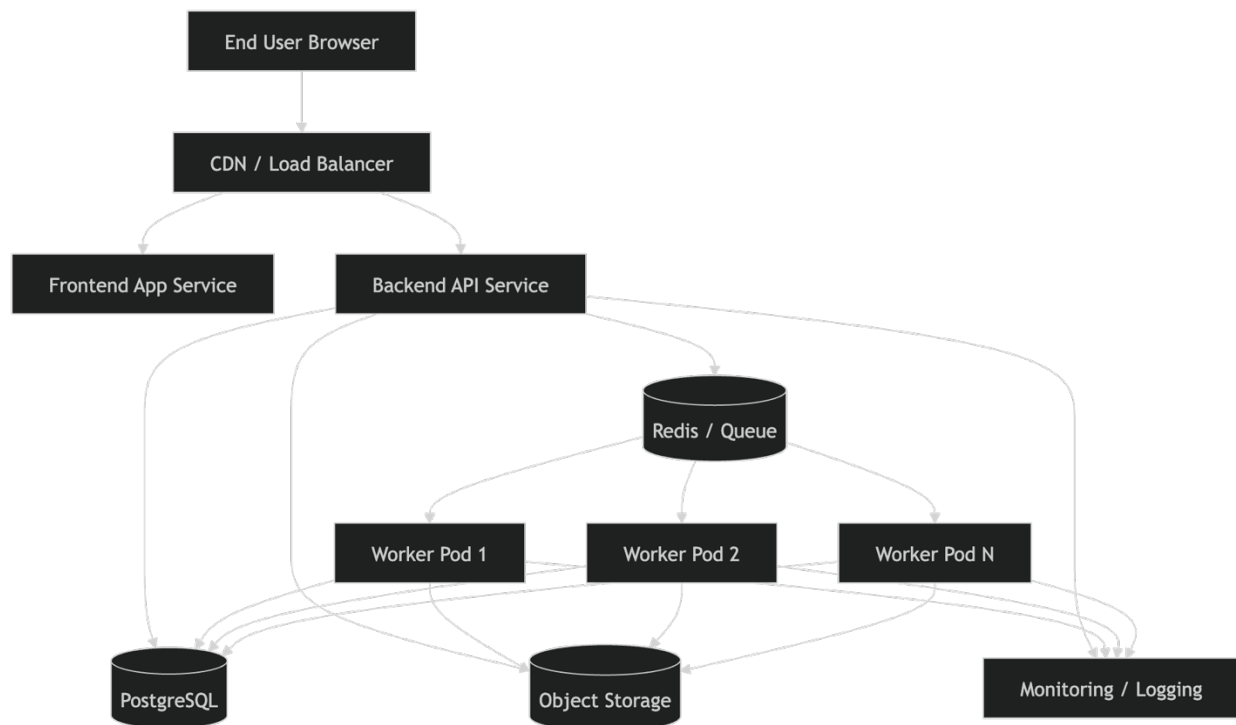
Explanation

A production deployment should separate stateless web services from compute-heavy worker services.

Suggested deployment blocks:

- CDN / Load Balancer
- Frontend app
- API service
- Worker autoscaling group
- PostgreSQL
- Redis/queue
- object storage
- monitoring stack

Mermaid Diagram



9. Data Flow Diagram (DFD) – Level 0

Explanation

This is the context-level DFD.

External entities:

- User
- Admin

Single main process:

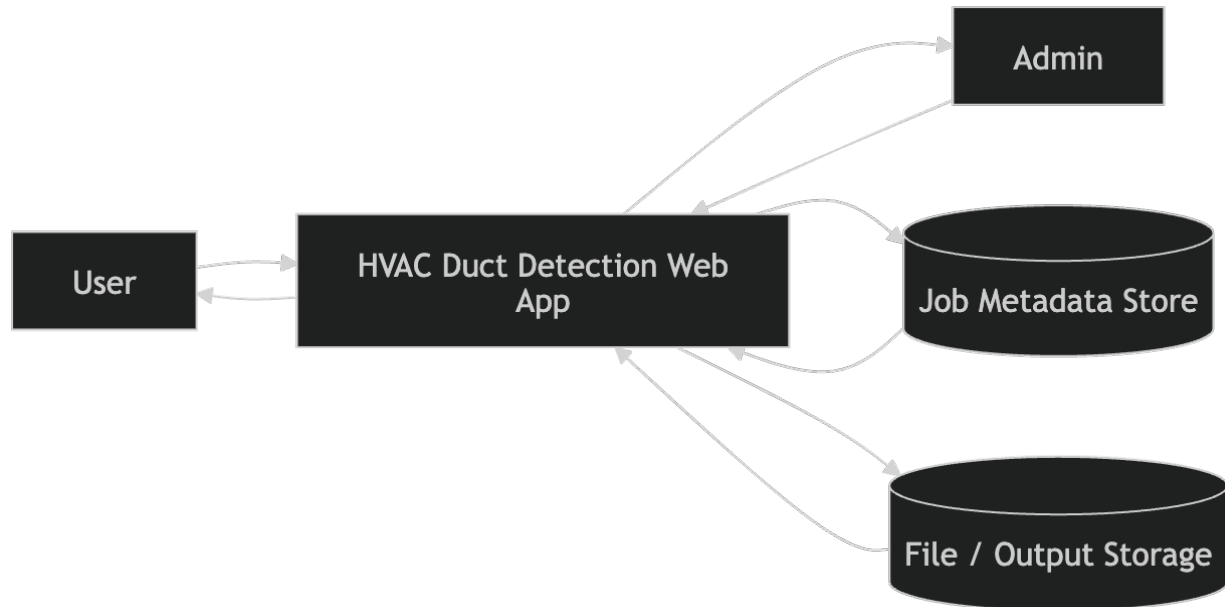
- HVAC Duct Detection Web App

Data stores:

- Job Metadata Store
- File Storage

High-Level Design (HLD)

Mermaid Diagram



10. Data Flow Diagram (DFD) – Level 1

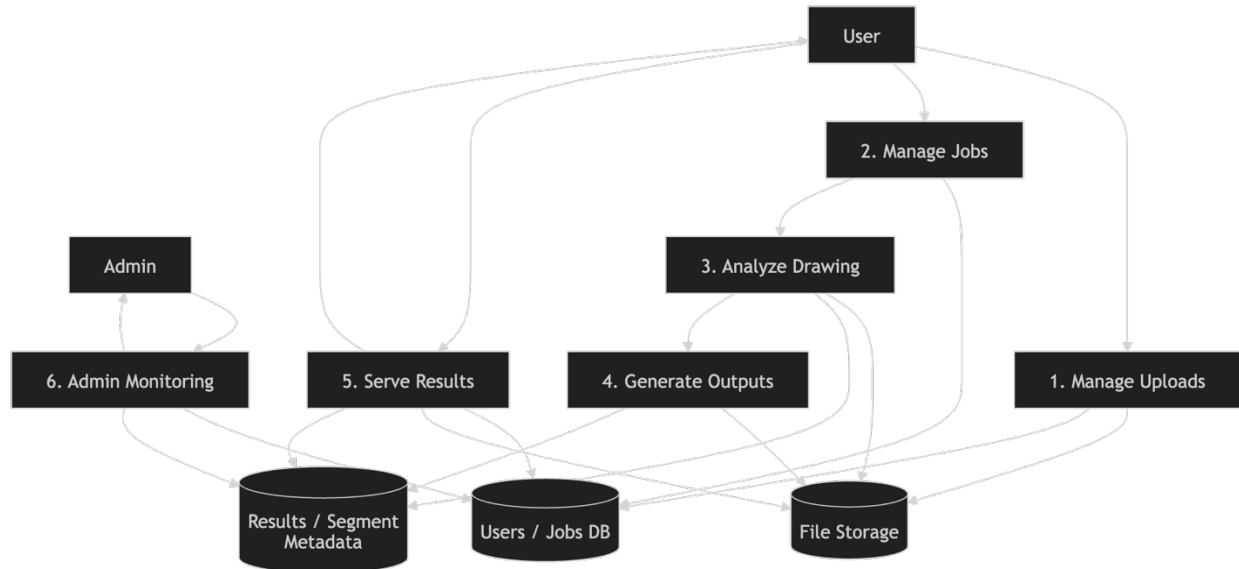
Explanation

This breaks the app into major internal processes:

1. Manage Uploads
2. Manage Jobs
3. Analyze Drawing
4. Generate Outputs
5. Serve Results

High-Level Design (HLD)

Mermaid Diagram



11. Data Flow Diagram (DFD) – Level 2 for Analyze Drawing

Explanation

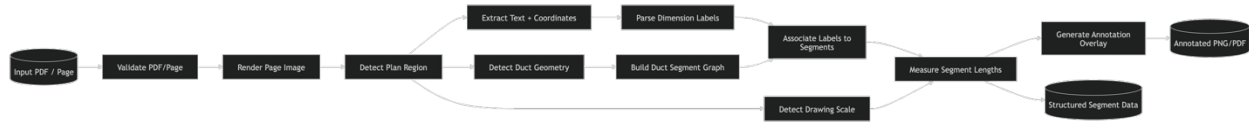
This is the system's core engineering process.

Subprocesses:

- validate PDF
- render page
- detect plan region
- extract text
- detect ducts
- build segments
- associate labels
- detect scale
- measure lengths
- annotate/export

High-Level Design (HLD)

Mermaid Diagram



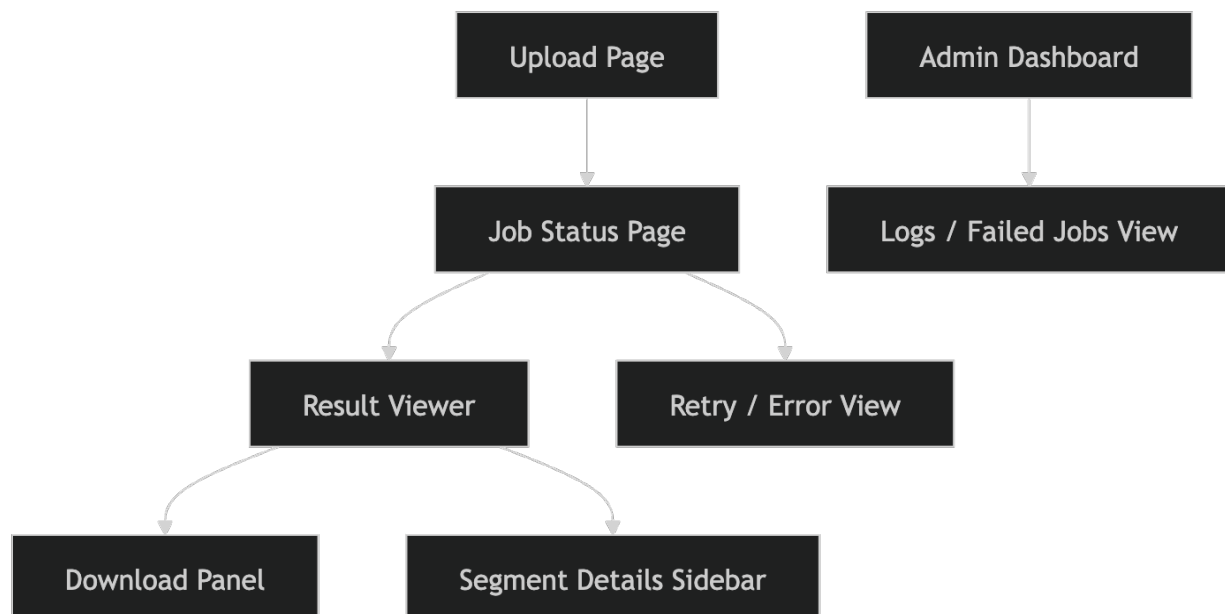
12. Subsystem HLD

12.1 Frontend Subsystem

Responsibilities

- login/session
- upload UI
- job status UI
- result preview
- download actions
- admin/debug view

Mermaid



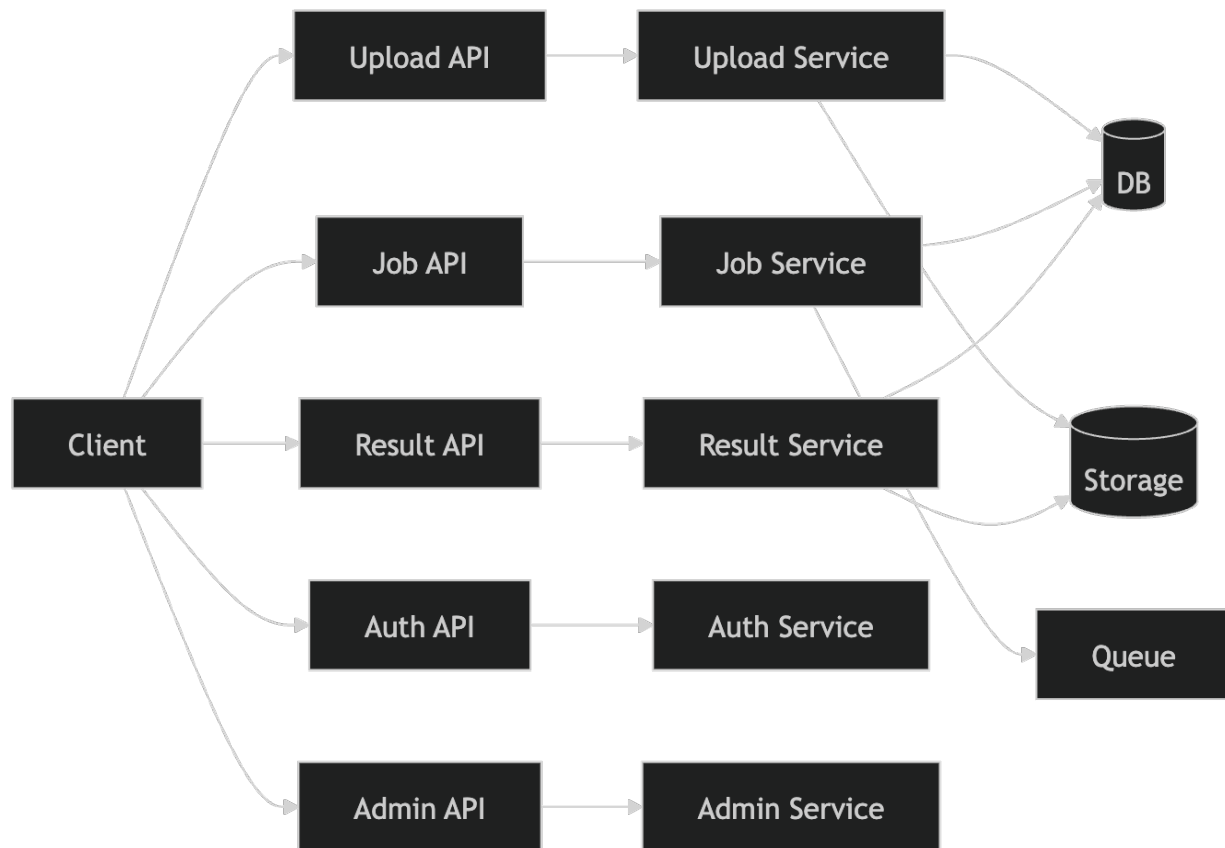
High-Level Design (HLD)

12.2 Backend/API Subsystem

Responsibilities

- auth checks
- uploads
- job lifecycle
- result retrieval
- admin/debug endpoints

Mermaid



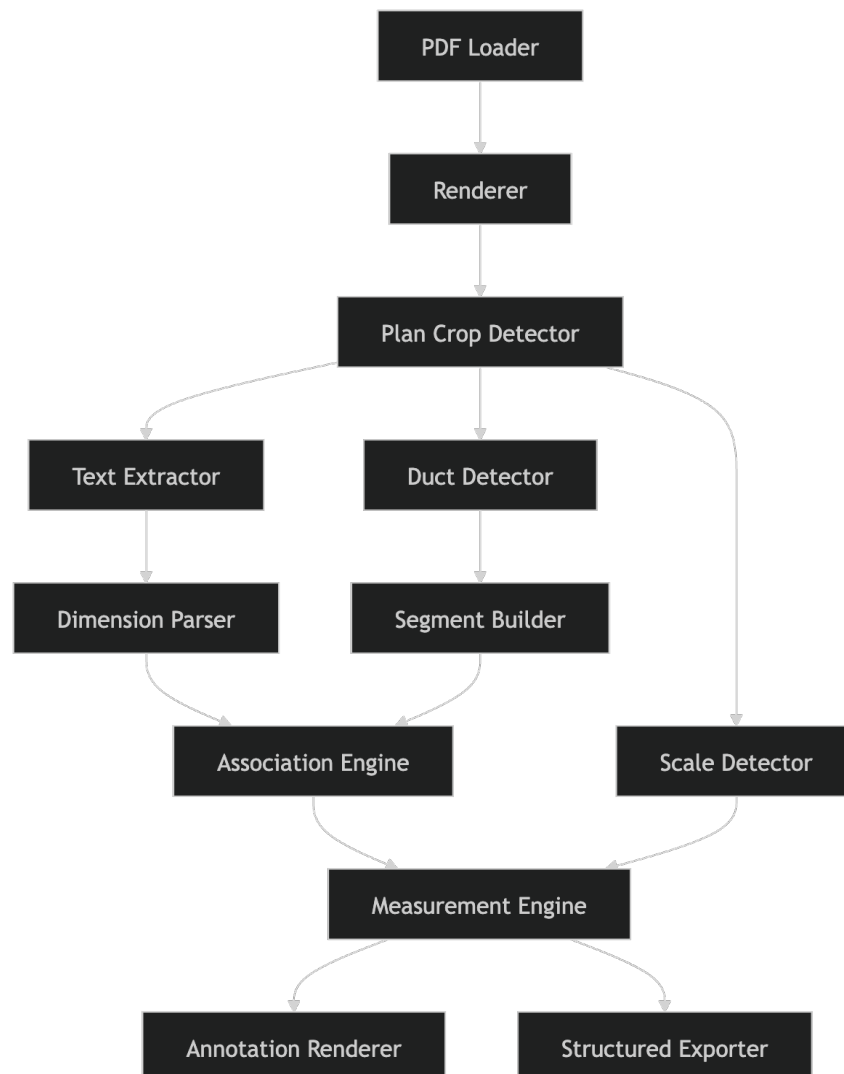
High-Level Design (HLD)

12.3 Processing Engine Subsystem

Responsibilities

- drawing parsing
- text extraction
- duct graph construction
- length estimation
- overlay generation

Mermaid



High-Level Design (HLD)

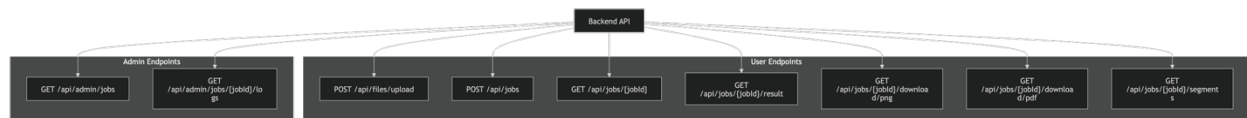
13. Suggested API HLD

Explanation

At high level, the system can expose these endpoints:

- POST /api/files/upload
- POST /api/jobs
- GET /api/jobs/{jobId}
- GET /api/jobs/{jobId}/result
- GET /api/jobs/{jobId}/download/png
- GET /api/jobs/{jobId}/download/pdf
- GET /api/jobs/{jobId}/segments
- GET /api/admin/jobs
- GET /api/admin/jobs/{jobId}/logs

Mermaid



14. Data Storage HLD

Explanation

Two main persistent stores are enough at HLD level:

Relational DB

Stores:

- users
- files
- jobs
- pages
- extracted segments

High-Level Design (HLD)

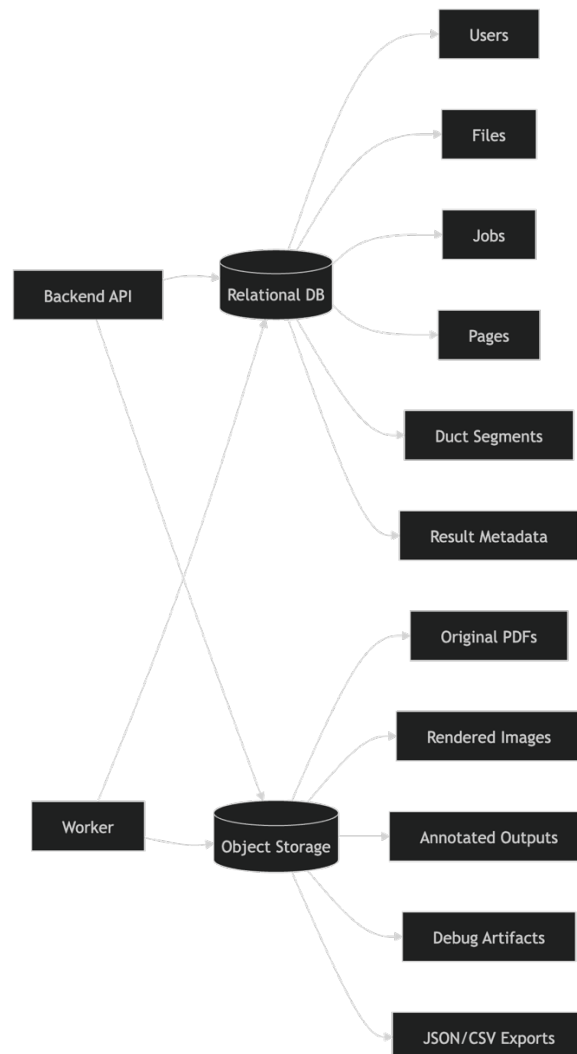
- result metadata
- logs references

Object Storage

Stores:

- original PDFs
- rendered page images
- debug images
- annotated PNG/PDF
- CSV/JSON exports

Mermaid



15. Key Architectural Decisions

15.1 Why async job processing

Drawing analysis is CPU-heavy and sometimes memory-heavy. Keeping it outside the request-response cycle avoids timeouts and makes UX better.

15.2 Why object storage

Annotated PNG/PDF files and intermediate images are large binary assets and should not live in relational tables.

15.3 Why modular processing engine

Future improvements may replace only one module, such as:

- duct detector
- OCR/text extractor
- scale calibrator
- annotation renderer

without rewriting the whole app.

15.4 Why separate metadata and file storage

This keeps the system scalable and operationally clean.

High-Level Design (HLD)

16. Recommended Tech Stack at HLD Level

A good web-app stack for this design would be:

- **Frontend:** React or Next.js
- **Backend API:** FastAPI or Node.js/NestJS
- **Workers:** Python workers
- **Queue:** Redis + Celery / RQ, or RabbitMQ / SQS
- **PDF & vision engine:** Python
- **DB:** PostgreSQL
- **Object storage:** S3-compatible storage
- **Monitoring:** Prometheus/Grafana or cloud-native monitoring
- **Deployment:** Docker + Kubernetes or ECS

Python is especially strong here because the core document-analysis pipeline depends on PDF and CV libraries.

17. End-to-End HLD Summary

In one sentence:

This system is a **web application with asynchronous document-analysis workers** that ingest HVAC mechanical drawing PDFs, isolate the plan area, extract text and duct geometry, associate dimensions and lengths, render annotated overlays, store outputs, and return previewable downloadable results to the user.